

On the Effectiveness of Domain Randomization in MuJoCo Locomotion Tasks of Increasing Complexity

1st Setare Derispour

dept. Control and Computer Engineering (DAUIN)
Politecnico di Torino
Turin, Italy
s344770@studenti.polito.it

2nd Francesco Masciari

dept. Control and Computer Engineering (DAUIN)
Politecnico di Torino
Turin, Italy
s319948@studenti.polito.it

Abstract—Domain randomization is widely used to improve robustness and generalization of reinforcement learning policies trained in simulation. However, its effectiveness is often discussed without explicitly considering the role of task complexity and training stability. In this work, we analyze the behavior of different domain randomization strategies — no domain randomization, uniform domain randomization (UDR), and adversarial domain randomization (ADR) — in simulated locomotion tasks of increasing complexity.

Starting from the Hopper environment used in the course laboratory, we extend the analysis to Walker2d, a more complex bipedal locomotion task, using mass randomization to introduce controlled domain shifts in a sim-to-sim setting. The goal of this study is not to maximize performance, but to investigate whether and how domain randomization strategies scale from simpler to more complex environments.

Our results indicate that while domain randomization can improve robustness in simpler scenarios, its benefits become less clear and more sensitive to training dynamics as task complexity increases. These observations highlight the importance of careful experimental design and realistic expectations when applying domain randomization techniques to complex control problems.

Index Terms—Reinforcement Learning, Domain Randomization, Adversarial Domain Randomization, Uniform Domain Randomization, MuJoCo, Hopper, Walker2d, Robustness, Sim-to-Real Transfer

I. INTRODUCTION

Reinforcement learning (RL) has shown strong potential for learning control policies directly from interaction, particularly in simulated environments where data collection is efficient and safe [1], [2]. However, policies trained under fixed and idealized simulation conditions often exhibit poor generalization when exposed to dynamics different from those seen during training, a phenomenon commonly referred to as the reality gap [3].

Domain randomization addresses this issue by training policies over a distribution of environment parameters rather than a single nominal model [4], [5]. While this approach is widely adopted and often reported as effective, its actual benefits may depend on several factors, including task complexity, parameter selection, and training stability [3]. In particular, it remains unclear whether more advanced strategies, such as adversarial

domain randomization (ADR), consistently provide advantages over simpler approaches like uniform domain randomization (UDR), especially as the underlying control problem becomes more complex.

In this project extension, we build upon the laboratory setup based on the Hopper environment, where domain randomization over mass parameters was introduced to simulate a controlled source–target mismatch [4], [5]. We extend this baseline in two directions. First, we incorporate ADR as an alternative to UDR, aiming to explore its behavior under the same experimental conditions. Second, we evaluate all randomization strategies on Walker2d, a more complex bipedal locomotion task, to study whether robustness gains observed in simpler environments persist as system complexity increases.

The objective of this work is not to redesign reward functions or to optimize peak performance, but to analyze and compare the behavior of different domain randomization strategies across environments. By focusing on learning dynamics and generalization trends rather than absolute scores, we aim to highlight both the potential and the limitations of domain randomization when applied to increasingly complex control problems.

II. RELATED WORK

Domain randomization has been widely adopted as a practical approach to improve the robustness and generalization of reinforcement learning policies trained in simulation, particularly in sim-to-real transfer scenarios [3]–[5]. By exposing the agent to variations in environment dynamics during training, domain randomization aims to reduce overfitting to a single nominal model and mitigate the effects of the reality gap.

Several works have explored uniform domain randomization, where environment parameters are sampled from fixed distributions, demonstrating improved robustness across a variety of control and manipulation tasks [4], [5]. More recently, adversarial domain randomization has been proposed as an adaptive alternative, in which a second agent actively selects environment parameters to challenge the learning policy. While this approach can accelerate robustness acquisition in

certain settings, it may also introduce additional instability into the training process [3].

In this work, we follow a sim-to-sim evaluation strategy to analyze the behavior of different domain randomization techniques under controlled domain shifts. Rather than focusing on real-world deployment, we aim to study how robustness trends evolve as task complexity increases, using locomotion benchmarks of increasing difficulty.

III. EXPERIMENTAL SETUP AND METHODOLOGY

A. Environments and Domains

1) *Hopper Environment*: Hopper is used as a baseline locomotion task due to its simple morphology and limited number of degrees of freedom, which allows controlled analysis of domain randomization effects under relatively stable learning dynamics.

2) *Walker2d Environment*: Walker2d is selected as a more complex bipedal locomotion task, introducing additional degrees of freedom and coordination requirements that make the learning process more sensitive to dynamics variations and training instability.

3) *Source and Target Domains*: This sim-to-sim setup is intended to emulate a controlled sim-to-real mismatch, allowing us to isolate the effects of domain randomization under increasing task complexity.

To evaluate the robustness and generalization capability of reinforcement learning policies, two distinct variants of each locomotion environment are defined: a *source* environment and a *target* environment. This separation enables the introduction of controlled domain shifts and allows policy performance to be assessed under altered dynamics.

The source environment represents the nominal simulation setting used during training. It is based on predefined physical parameters derived from the MuJoCo model. In contrast, the target environment differs from the source through modifications of selected physical properties, while preserving the task definition and reward structure. This design introduces a dynamics mismatch between training and evaluation conditions.

For the Hopper environment, the domain shift is introduced by modifying the torso mass in the source environment, while keeping the remaining link masses unchanged. Table I reports the mass values used in the source and target Hopper environments. This variation affects the center of mass and inertial properties of the robot, altering its balance and locomotion dynamics.

In the Walker2d environment, a similar source–target distinction is applied by modifying the mass distribution of selected body links. Due to the bipedal structure of Walker2d and the increased number of degrees of freedom, such mass variations introduce more pronounced changes in gait coordination and stability. The corresponding mass parameters for the Walker2d source and target environments are summarized in the Table II .

4) *Episode Termination Condition*: During training, policies are optimized from finite-length trajectories generated by interactions with the environment. Episode termination

TABLE I: Mass Values in Source and Target Hopper Environments

Body Part	Source Mass (kg)	Target Mass (kg)
Torso	2.534	3.534
Thigh	3.927	3.927
Leg	2.714	2.714
Foot	5.089	5.089

TABLE II: Mass Values in Source and Target Walker2d Environments

Body Part	Source Mass (kg)	Target Mass (kg)
Torso	2.665	3.665
Thigh (each)	4.058	4.058
Leg (each)	2.781	2.781
Foot (each)	3.167	3.167

therefore plays a practical role in shaping rollout statistics, learning stability, and the interpretation of performance metrics such as episode length and cumulative reward.

Episodes terminate when standard environment health conditions are violated or when the maximum episode length is reached. Termination occurs if the robot loses balance, if the torso configuration exits predefined healthy bounds, or if any state variable becomes non-finite. For the Walker2d environment, minor adjustments to termination thresholds were introduced to avoid excessively short rollouts during early training and to improve learning stability, while preserving the original task definition.

B. Training Setup

All experiments are conducted using Proximal Policy Optimization (PPO) [6], as provided in the baseline code, without modifications to the learning algorithm. Training is performed entirely in simulation using vectorized environments in order to improve sample efficiency.

Each policy is trained for a fixed budget of timesteps using a single random seed. While this choice limits statistical significance, it allows us to focus on qualitative learning dynamics and robustness trends, which are the primary objectives of this study. For this reason, results are interpreted qualitatively rather than statistically.

Policy evaluation is performed periodically during training and at convergence by measuring average episodic returns over multiple evaluation episodes. Performance is reported both in the source environment and in the target environment to assess generalization under controlled dynamics mismatch.

IV. DOMAIN RANDOMIZATION METHODS

Domain randomization is employed to reduce the sim-to-real gap by exposing the learning agent to variations in environment dynamics during training [3]–[5]. As a reference baseline, policies trained without domain randomization rely on a fixed and deterministic dynamics model, where all physical parameters remain constant and correspond to the source environment. While this setup allows efficient learning under nominal conditions, it often leads to policies that overfit

to the source dynamics and generalize poorly when evaluated in a target environment with different physical properties [3].

To address this limitation, we investigate two domain randomization techniques: Uniform Domain Randomization (UDR) and Adversarial Domain Randomization (ADR). Both methods introduce variability in selected physical parameters during training, with the goal of improving robustness and transfer performance. These techniques are applied consistently across both the Hopper and Walker2d environments.

A. Uniform Domain Randomization

Uniform Domain Randomization (UDR) introduces variability in the environment dynamics by sampling selected physical parameters from fixed uniform distributions at the beginning of each training episode [4]. Instead of training on a single deterministic model, the agent is exposed to a distribution of environments, encouraging the learning of policies that are less sensitive to precise parameter values.

In this work, UDR is implemented by randomizing the masses of specific robot links while keeping the overall morphology unchanged. For the Hopper environment, the masses of the thigh, leg, and foot are randomized. For the Walker2d environment, the same mass randomization is applied symmetrically to both legs, including the thigh, leg, and foot segments on the left and right sides. In both environments, mass values are sampled uniformly within a range defined as $\pm 20\%$ of their nominal values.

The randomization is applied once per episode reset, ensuring that each episode is executed under a consistent but randomly sampled set of dynamics. This approach improves robustness by reducing overfitting to a single dynamics configuration. However, the effectiveness of UDR depends on the choice of the randomization range, which must balance sufficient variability with training stability [3].

B. Adversarial Domain Randomization

Adversarial Domain Randomization (ADR) extends the concept of domain randomization by adaptively adjusting the range of randomized parameters based on the agent’s performance during training [5]. Rather than sampling parameters uniformly from a fixed range, ADR focuses training on increasingly challenging dynamics that are more likely to degrade policy performance.

In the implemented ADR framework, the randomization range is controlled by a scale parameter that is updated at the end of each episode. If the episode return exceeds a predefined success threshold, the scale is increased, thereby expanding the range of randomized masses and making the environment more challenging. Conversely, if performance falls below the threshold, the scale is reduced to preserve training stability. The scale is constrained within predefined minimum and maximum bounds to avoid excessively small or large perturbations.

ADR is applied to the same set of body masses as UDR in both environments. For Hopper, the thigh, leg, and foot masses are randomized, while for Walker2d, mass randomization is

applied symmetrically to both legs. Compared to UDR, ADR prioritizes training on difficult regions of the parameter space rather than uniformly exploring all variations.

While this targeted strategy can lead to improved robustness and transfer performance, it also introduces additional complexity in the training dynamics, as highlighted in prior work [3].

V. RESULTS AND DISCUSSION

A. Hopper Results

Table III summarizes the evaluation results for Hopper with a training budget of 1M timesteps. When trained without domain randomization (NoDR), the policy achieves high performance on the source domain but exhibits a substantial drop when evaluated on the target domain, resulting in a large performance gap between $S \rightarrow S$ and $S \rightarrow T$. This behavior indicates strong sensitivity to the introduced mass shift and limited robustness.

The training reward curves in Fig. 1 (top row) show that learning in Hopper is fast and stable across all strategies, reflecting the limited complexity of the task. Uniform Domain Randomization (UDR) slightly improves training stability and source-domain performance; however, its transfer performance to the target domain remains limited. In fact, the $S \rightarrow T$ reward under UDR is lower than that obtained with NoDR, leading to an even larger domain gap. This suggests that uniformly sampling parameters from a fixed randomization range does not necessarily improve generalization when the sampled distribution is not well aligned with the target dynamics.

In contrast, Adversarial Domain Randomization (ADR) exhibits a different trade-off. While the average reward is lower and variance is higher, ADR achieves nearly identical performance on source and target domains, effectively minimizing the domain gap. This robustness-oriented behavior is further explained by the evolution of the ADR scale shown in Fig. 2, which becomes active early during training, and by the progressive expansion of ADR-driven mass perturbations in Fig. 4a. These plots indicate that, in Hopper, a minimally competent locomotion policy is learned quickly, allowing the adaptive mechanism to expand the randomized domain within the available training budget.

Overall, Hopper acts as a sanity check scenario in which training curves, ADR internal dynamics, and evaluation results are consistent: UDR stabilizes learning but does not guarantee improved transfer, while ADR actively reduces sensitivity to domain shifts once basic policy competence is achieved.

B. Walker2d Results

Results for Walker2d highlight the increased difficulty of applying domain randomization in more complex control tasks. Table IV reports evaluation metrics obtained with a training budget of 1M timesteps. In this regime, both NoDR and UDR achieve relatively high rewards on both source and target domains, resulting in a small performance gap. This indicates that, for the considered mass shift, Walker2d policies can

tolerate moderate dynamics variations even without adaptive mechanisms.

However, the training reward curves in Fig. 1 (bottom row) reveal substantially higher variance and slower convergence compared to Hopper, reflecting the increased coordination demands of bipedal locomotion. In this setting, ADR performs significantly worse at 1M timesteps, achieving low rewards on both domains and failing to reduce the source–target gap.

The underlying reason for this behavior is clarified by the ADR scale evolution in Fig. 3. With a training budget of 1M timesteps, the ADR scale remains close to its minimum value throughout training, indicating that the adaptive mechanism does not become active. As a consequence, the randomization range does not expand, and ADR effectively reduces to near-deterministic training. The absence of meaningful mass perturbations confirms that the policy has not yet reached the competence threshold required for adversarial adaptation.

Motivated by this observation, training is extended to 3M timesteps. As shown in Table V, UDR achieves the highest and most stable performance, with nearly identical rewards on source and target domains and reduced variance. ADR, while still under-performing UDR in terms of absolute reward and exhibiting higher variance, no longer collapses and shows a substantially reduced domain gap compared to the 1M regime. This improvement coincides with the activation and stabilization of the ADR scale (Fig. 3) and the expansion of ADR-driven mass perturbations (Fig. 4b), indicating that the policy has finally reached sufficient competence for adaptive randomization to become effective.

These results confirm that, in Walker2d, ADR requires a significantly larger training budget to operate as intended, reflecting the increased complexity of the control problem and the delayed emergence of stable locomotion strategies.

C. Cross-Environment Analysis

A comparison between Hopper and Walker2d reveals that the effectiveness of domain randomization strategies is strongly modulated by task complexity. Despite applying an identical torso mass shift in both environments, the resulting impact on learning dynamics, internal ADR behavior, and transfer performance differs markedly due to differences in morphology and coordination requirements.

In Hopper, the rapid emergence of a competent policy enables early activation of ADR, leading to a near-complete reduction of the domain gap within a limited training budget. In Walker2d, the same perturbation delays policy competence, preventing ADR from becoming effective until significantly more training data are provided. In this more complex setting, UDR emerges as a more reliable and computationally efficient baseline, while ADR offers robustness benefits only after prolonged training and at the cost of increased variance.

VI. CONCLUSION

In this work, we analyzed the behavior of domain randomization strategies under increasing task complexity using the Hopper and Walker2d MuJoCo locomotion environments. By

comparing NoDR, UDR, and ADR in a controlled sim-to-sim setting, we focused not only on final performance but also on how robustness to domain shifts emerges as a function of both training budget and environment complexity.

Our results show that in simple tasks such as Hopper, characterized by low-dimensional dynamics and limited coordination requirements, Adversarial Domain Randomization can effectively reduce the source–target performance gap within a limited training budget, even at the cost of lower average rewards. In more complex environments like Walker2d, which require coordinated bipedal control, ADR requires substantially more training data before becoming active, as a minimally competent policy must first be learned for the adaptive mechanism to expand the randomized domain. In this regime, Uniform Domain Randomization provides a strong and reliable baseline, while ADR offers robustness benefits only after prolonged training.

Overall, these findings highlight that domain randomization is not a plug-and-play solution. Its effectiveness depends critically on task complexity, training budget, and policy competence. Rather than aiming solely at maximizing reward, domain randomization techniques should be evaluated in terms of their ability to reduce sensitivity to domain shifts under realistic computational and task constraints.

REFERENCES

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [2] J. Kober, J. A. Bagnell, and J. Peters, “Reinforcement learning in robotics: A survey,” *The International Journal of Robotics Research*, vol. 32, no. 11, pp. 1238–1274, 2013.
- [3] S. Höfer, K. E. Bekris, A. Handa, J. C. Gamboa, F. Golemo, M. Mozifian, and M. White, “Perspectives on sim-to-real transfer for robotics: A summary of the r:ss 2020 workshop,” *arXiv preprint arXiv:2012.06104*, 2020.
- [4] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain randomization for transferring deep neural networks from simulation to the real world,” *arXiv preprint arXiv:1703.06907*, 2017.
- [5] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, “Sim-to-real transfer of robotic control with dynamics randomization,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2018, pp. 3803–3810.
- [6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.

TABLE III: Hopper (training budget: 1M timesteps). Policies are trained on the source domain and evaluated on both source (S→S) and target (S→T).

DR Strategy	Domain	Avg. Reward	Std. Dev.
No DR	S→S	1620.17	25.63
No DR	S→T	1027.92	13.12
UDR	S→S	1711.08	32.43
UDR	S→T	869.85	8.56
ADR	S→S	1499.67	143.47
ADR	S→T	1477.58	182.09

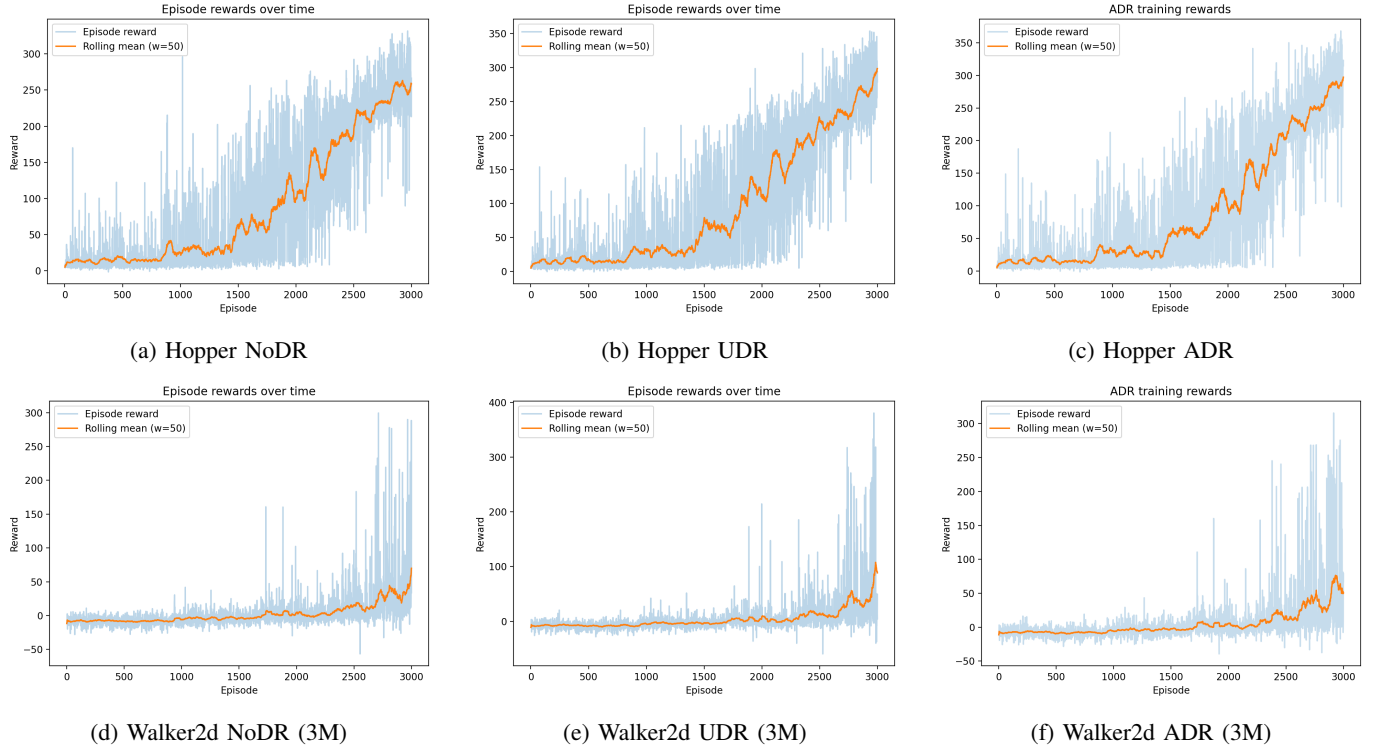


Fig. 1: Training reward curves under NoDR, UDR, and ADR. Hopper results (top row) show stable learning within a limited training budget, while Walker2d results (bottom row) highlight increased variance and the need for extended training to achieve comparable policy competence.

TABLE IV: Walker2d (training budget: 1M timesteps). Policies are trained on the source domain and evaluated on both source ($S \rightarrow S$) and target ($S \rightarrow T$).

DR Strategy	Domain	Avg. Reward	Std. Dev.
No DR	$S \rightarrow S$	2239.29	348.82
No DR	$S \rightarrow T$	2398.32	228.70
UDR	$S \rightarrow S$	2275.74	19.77
UDR	$S \rightarrow T$	2372.02	21.00
ADR	$S \rightarrow S$	896.72	71.68
ADR	$S \rightarrow T$	763.90	27.22

TABLE V: Walker2d (training budget: 3M timesteps). Policies are trained on the source domain and evaluated on both source ($S \rightarrow S$) and target ($S \rightarrow T$).

DR Strategy	Domain	Avg. Reward	Std. Dev.
No DR	$S \rightarrow S$	3481.08	962.86
No DR	$S \rightarrow T$	3382.10	1024.50
UDR	$S \rightarrow S$	3875.00	103.49
UDR	$S \rightarrow T$	3854.32	388.23
ADR	$S \rightarrow S$	2532.17	864.44
ADR	$S \rightarrow T$	2376.51	977.26

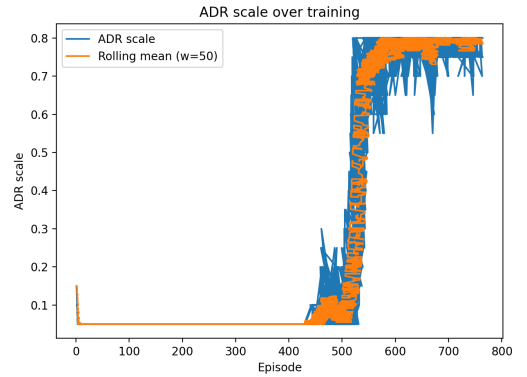


Fig. 2: ADR scale schedule in Hopper (1M steps). The scale controls the magnitude of mass randomization and is adapted based on policy performance.

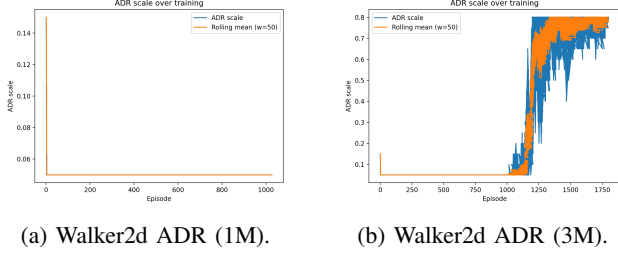
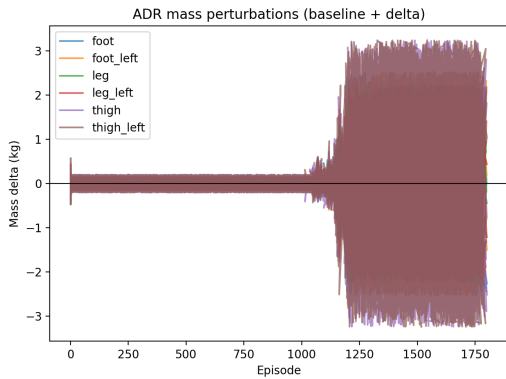
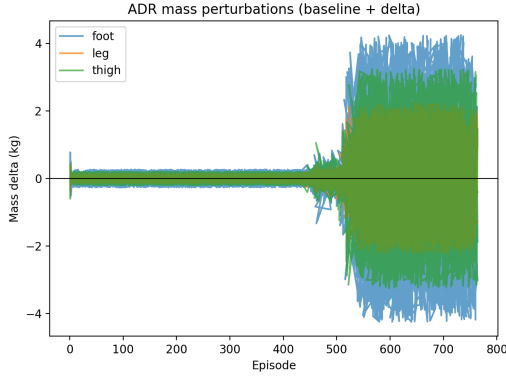


Fig. 3: ADR scale evolution in Walker2d for two training budgets. With 1M timesteps, the scale stays near its minimum, suggesting that ADR does not become active. With 3M timesteps, the scale expands and then stabilizes, indicating that ADR requires a minimally competent policy before it can effectively widen the randomized domain.



(b) ADR-driven mass perturbation magnitude in the Walker2d environment (3M timesteps).

Fig. 4: The perturbation range progressively expands during training, indicating that the ADR mechanism becomes active once a basic locomotion policy is learned.